Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 4
по дисциплине «Типы и структуры данных»

Тема Разреженные матрицы

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Кольцевая схема Рейнбольдта — Местеньи 5

2 Конструкторская часть **6**

2.1 Структура CompressedMatrix 6

2.2 Алгоритмы сжатия и распаковки разреженной матрицы 6

2.3 Алгоритмы транспонирования и вычисления среднего арифметического сжатой разреженной матрицы 9

3 Технологическая часть **12**

3.1 Реализация и выбор средств 12

3.2 Тестирование программы 12

ЗАКЛЮЧЕНИЕ **15**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **16**

ПРИЛОЖЕНИЕ А **17**

ВВЕДЕНИЕ

В современных условиях разработки программных алгоритмов часто приходится работать с матрицами, содержащими преимущественно нулевые элементы. Для эффективного хранения и обработки таких структур применяются специальные форматы разреженных матриц, которые позволяют существенно экономить память и ускорять вычисления, храня только ненулевые элементы и их позиции, что особенно важно при работе с крупномасштабными задачами.

Целью работы является получить практический навык работы с разреженными матрицами. Для достижения поставленной задачи необходимо выполнить следующие пункты:

1) описать схему сжатого представления разреженных матриц, согласно индивидуальному варианту;

2) разработать алгоритмы упаковки и распаковки матрицы, а также алгоритмы, выполняющие операции над сжатыми матрицами без распаковки согласно варианту;

3) описать разработанные алгоритмы псевдокодом;

4) реализовать описанные алгоритмы;

5) провести тестирование реализации разработанных алгоритмов.

1 Аналитическая часть

Разреженные матрицы — это матрица, подавляющее количество элементов которой ($> 90\%$) является нулями.

1.1 Кольцевая схема Рейнбольдта — Местеньи

Кольцевая схема Рейнбольдта — Местеньи — это модификация схемы **Кнута**, сохраняющая ее ценные свойства, но использующая значительно меньше накладной памяти.

Ненулевые элементы разреженной матрицы хранятся в компактной форме и в произвольном порядке в одномерном массиве `AN`.

Чтобы можно было легко отыскивать элементы произвольной строки или столбца матрицы, необходимы указатели входа для строк и столбцов, сообщающие начало каждого строчного или столбцового списка. Пусть `NR` — массив, хранящий строчные указатели, а `NC` — массив столбцовых указателей. Связные списки строк и столбцов закольцовываются, а начальные позиции списков включаются в указатели входа. Списки, ассоциированные со строками (столбцами), попарно не пересекаются и потому могут быть совместно хранимы одним массивом `NR` (для столбцов `NC`).

Массивы `AN`, `NR` и `NC` имеют одинаковую длину, и их одноименные позиции соответствуют друг другу. Пусть `JR` и `JC` — массивы, содержащие указатели входа для строк и столбцов, расположенные в соответствии с порядком строк и столбцов матрицы.

Эта схема более плотная по сравнению со схемой **Кнута**. Однако нужно уяснить, что если приходится просматривать элементы некоторой строки (или столбца), то мы не получаем никакой информации о столбцовых (строчных) индексах этих элементов [1].

Вывод

Были приведены все необходимые определения, а также описана **кольцевая схема Рейнбольдта — Местеньи**.

2 Конструкторская часть

В данном разделе представлены описания с помощью псевдокода алгоритмов сжатия и распаковки разреженных матриц, хранящихся по схеме Рейнбольдта — Местеньи, а также алгоритмов транспонирования и вычисления среднего арифметического сжатой матрицы. Для написания псевдокода использовался пакет `algorithmicx` для `latex` [2].

2.1 Структура CompressedMatrix

Структура **CompressedMatrix** предназначена для хранения сжатого представления матрицы по схеме Рейнбольдта — Местеньи и содержит следующие поля:

— целое число `rows` — количество строк матрицы;

— целое число `cols` — количество столбцов матрицы;

— массив вещественных чисел `an` — ненулевые элементы матрицы;

— массив целых чисел `nr` — строчные указатели на элементы в `an`;

— массив целых чисел `nc` — столбцовые указатели на элементы в `an`;

— массив целых чисел `jr` — индексы первых ненулевых элементов строк в `an`;

— массив целых чисел `jc` — индексы первых ненулевых элементов столбцов в `an`.

2.2 Алгоритмы сжатия и распаковки разреженной матрицы

Алгоритм сжатия разреженной матрицы

Ниже представлено описание алгоритма сжатия разреженной матрице с помощью псевдокода.

```
procedure compress(original_matrix)
    result ← новый объект CompressedMatrix
    result.rows ← размер original_matrix
    result.cols ← размер original_matrix[0]
    result.jr ← вектор размера rows, инициализированный -1
    result.jc ← вектор размера cols, инициализированный -1
    rowLinks ← вектор векторов размера rows
    colLinks ← вектор векторов размера cols
```

Рисунок 2.1 — Алгоритм сжатия матрицы, часть 1

```
    for i ← 0 до rows − 1 do
        for j ← 0 до cols − 1 do
            val ← original_matrix[i][j]
            if val ≠ 0 then
                index ← размер result.an
                добавить val в result.an
                добавить index в rowLinks[i]
                добавить index в colLinks[j]
            end if
        end for
    end for
    result.nr ← вектор размера an.size, инициализированный -1
    result.nc ← вектор размера an.size, инициализированный -1
    Обработка строковых связей:
    for i ← 0 до rows − 1 do
        if rowLinks[i] не пуст then
            result.jr[i] ← первый элемент rowLinks[i]
            for k ← 0 до размер rowLinks[i] − 1 do
                current_idx ← rowLinks[i][k]
                if k + 1 < размер rowLinks[i] then
                    next_idx ← rowLinks[i][k + 1]
                else
                    next_idx ← rowLinks[i][0]
                end if
                result.nr[current_idx] ← next_idx
            end for
        end if
    end for
    Обработка столбцовых связей:
    for j ← 0 до cols − 1 do
        if colLinks[j] не пуст then
            result.jc[j] ← первый элемент colLinks[j]
            for k ← 0 до размер colLinks[j] − 1 do
                current_idx ← colLinks[j][k]
                if k + 1 < размер colLinks[j] then
                    next_idx ← colLinks[j][k + 1]
                else
                    next_idx ← colLinks[j][0]
                end if
                result.nc[current_idx] ← next_idx
            end for
        end if
    end for
    return result
end procedure
```

Рисунок 2.2 — Алгоритм сжатия матрицы, часть 2

Алгоритм распаковки матрицы

Ниже представлено описание алгоритма распаковки разреженной матрицы, хранящейся в сжатом виде, с помощью псевдокода.

```
procedure decompress(comp)
    result ← матрица размера rows × cols, заполненная нулями
    Обработка строк:
    for i ← 0 до comp.rows − 1 do
        if comp.jr[i] ≠ −1 then
            current ← comp.jr[i]
            start ← current
            Обход элементов строки:
            repeat
                j ← −1
                Поиск столбца для текущего элемента:
                for col ← 0 до comp.cols − 1 и пока j = −1 do
                    if comp.jc[col] ≠ −1 then
                        col_current ← comp.jc[col]
                        col_start ← col_current
                        Обход элементов столбца:
                        repeat
                            if col_current = current then
                                j ← col
                                прервать цикл
                            end if
                            col_current ← comp.nc[col_current]
                        until col_current = col_start
                    end if
                end for
                if j ≠ −1 then
                    result[i][j] ← comp.an[current]
                end if
                current ← comp.nr[current]
            until current = start
        end if
    end for
    return result
end procedure
```

Рисунок 2.3 — Алгоритм распаковки матрицы

2.3 Алгоритмы транспонирования и вычисления среднего арифметического сжатой разреженной матрицы

Алгоритм транспонирования сжатой разреженной матрицы

Ниже представлено описание алгоритма транспонирования разреженной матрицы, хранящейся в сжатом виде, с помощью псевдокода.

```
procedure transpose(original_matrix)
    if A.rows ≠ A.cols then
        выбросить ошибку "Матрица должна быть квадратной"
    end if
    n ← A.rows
    T ← новый объект CompressedMatrix
    T.rows ← n, T.cols ← n
    Определение столбцов для элементов
    element_col ← вектор размера A.an.size(), инициализированный −1
    for j ← 0 до n − 1 do
        if A.jc[j] ≠ −1 then
            current ← A.jc[j]
            first ← current
            repeat
                element_col[current] ← j
                current ← A.nc[current]
            until current = first или current = −1
        end if
    end for
    Сбор элементов для транспонированной матрицы
    temp_data ← вектор из n списков пар (столбец, значение)
    for i ← 0 до n − 1 do
        if A.jr[i] ≠ −1 then
            current ← A.jr[i]
            first ← current
            repeat
                j ← element_col[current]
                if j ≠ −1 then
                    value ← A.an[current]
                    добавить (i, value) в temp_data[j]
                end if
                current ← A.nr[current]
            until current = first или current = −1
        end if
    end for
```

Рисунок 2.4 — Алгоритм транспонирования сжатой разреженной матрицы, часть 1

```
    nnz ← A.an.size()
    T.an ← вектор размера nnz
    T.nr ← вектор размера nnz
    T.nc ← вектор размера nnz
    T.jr ← вектор размера n, инициализированный −1
    T.jc ← вектор размера n, инициализированный −1
    Заполнение строковых связей
    idx ← 0
    for i ← 0 до n − 1 do
        if temp_data[i] не пуст then
            T.jr[i] ← idx
            for k ← 0 до размер temp_data[i] − 1 do
                T.an[idx] ← temp_data[i][k].second
                T.nc[idx] ← temp_data[i][k].first
                if k < размер temp_data[i] − 1 then
                    T.nr[idx] ← idx + 1
                else
                    T.nr[idx] ← T.jr[i]
                end if
                idx ← idx + 1
            end for
        end if
    end for
    Построение столбцовых связей
    col_elements ← вектор из n списков индексов
    for pos ← 0 до nnz − 1 do
        col ← T.nc[pos]
        добавить pos в col_elements[col]
    end for
    for j ← 0 до n − 1 do
        if col_elements[j] не пуст then
            T.jc[j] ← col_elements[j][0]
            for k ← 0 до размер col_elements[j] − 1 do
                elem_idx ← col_elements[j][k]
                if k < размер col_elements[j] − 1 then
                    T.nc[elem_idx] ← col_elements[j][k + 1]
                else
                    T.nc[elem_idx] ← T.jc[j]
                end if
            end for
        end if
    end for
    return T
end procedure
```

Рисунок 2.5 — Алгоритм транспонирования сжатой разреженной матрицы, часть 2

Алгоритм вычисления среднего арифметического сжатой разреженной матрицы

Ниже представлено описание алгоритма вычисления среднего арифметического разреженной матрицы, хранящейся в сжатом виде, с помощью псевдокода.

```
procedure arithmetic_mean(comp)
    sum ← 0
    for каждого num в comp.an do
        sum ← sum + num
    end for
    result ← sum/(comp.cols × comp.rows)
    return result
end procedure
```

Рисунок 2.6 — Алгоритм вычисления среднего арифметического сжатой разреженной матрицы

Вывод

Была разработана структура **CompressedMatrix** для хранения матрицы в сжатом виде, а также описаны алгоритмы для работы с разреженными матрицами в сжатом виде согласно варианту.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации использовалась среда разработки $Visual\ Studio\ Code$ [3]. Язык программирования, на котором была выполнена реализация алгоритмов — $C++$. Реализация программы содержит структуру `CompressedMatrix` для хранения разреженной матрицы в сжатом виде, а также методы, описанные в предыдущем разделе. В листинге А.1 представлена программная реализация описанных алгоритмов и структуры.

3.2 Тестирование программы

Программа принимает размеры и элементы матрицы из файла. Была реализована функция `readMatrixFromFile`, предназначенная для считывания матрицы из файла и проверяющая корректность считываемых данных: элементы и размеры матрицы должны быть типа `integer` и указанное количество строк и столбцов должно совпадать с фактическим. Для проверки работы реализаций описанных алгоритмов сжатий был выбран текстовый файл, содержащий разреженную матрицу 10×10 (то есть не более десяти элементов ненулевые). В листинге 3.1 приведено содержимое данного файла.

```
10 10
0 0 0 8 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
4 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 6 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 3 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 7
0 1 0 0 0 0 0 0 0 0
```

Листинг 3.1 — Содержимое файла

После работы с файлами программа загружает все полученные результаты в новый текстовый файл. В листинге 3.2 приведено его содержимое после работы с матрицей, показанной выше.

```
Исходная матрица:
0 0 0 8 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
4 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 6 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 3 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 7
0 1 0 0 0 0 0 0 0 0

Сжатая матрица:
AN: 8 1 4 6 3 7 1
JR: 0 1 2 -1 3 -1 4 -1 5 6
JC: 2 6 -1 0 -1 3 -1 4 1 5
NR: 0 1 2 3 4 5 6
NC: 0 1 2 3 4 5 6

Распакованная матрица:
0 0 0 8 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0
4 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 6 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 3 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 7
0 1 0 0 0 0 0 0 0 0

Среднее арифметическое матрицы: 0.3

Сжатая транспонированная матрица:
AN: 4 1 8 6 3 1 7
JR: 0 1 -1 2 -1 3 -1 4 5 6
JC: 2 5 0 -1 3 -1 4 -1 6 1
NR: 0 1 2 3 4 5 6
NC: 0 1 2 3 4 5 6
```

```
Распакованная транспонированная матрица:
0 0 4 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 0 0 0
8 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 6 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 3 0 0 0
0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 7 0
```

Листинг 3.2 — Пример работы программы

Вывод

Программа успешно прошла тестирование на работоспособность.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы, а именно реализованы алгоритмы для работы с разреженными матрицами.

Также были выполнены все задачи:

1) описана схема Рейнбольдта — Местеньи сжатого представления разреженных матриц;

2) разработаны алгоритмы упаковки и распаковки матрицы, а также алгоритмы транспонирования матрицы и поиска среднего арифметического без распаковки;

3) разработанные алгоритмы описаны псевдокодом;

4) реализованы описанные алгоритмы;

5) проведено тестирование реализаций разработанных алгоритмов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Писсанецки С. — Технология разреженных матриц. — Мир, 1988.
2. Псевдокод алгоритма в TeX [Электронный ресурс]. URL: https://pro-prof.com/forums/topic/псевдокод-алгоритма-в-tex (дата обра- щения: 8 ноября 2025).
3. Visual Studio Code documentation. [Электронный ресурс]. URL: https://code.visualstudio.com/docs (дата обращения: 6 ноября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <stdexcept>
#include <unordered_map>
#include <iomanip>
#include <sstream>
#include <string>

using namespace std;

// Структура для хранения сжатой прямоугольной матрицы по схеме Рейнбольдта
    и Местеньи
struct CompressedMatrix {
  int rows;     // количество строк
  int cols;     // количество столбцов

  vector<double> an;  // ненулевые элементы
  vector<int> nr;     // индексы следующих элементов в той же строке
  vector<int> nc;     // индексы следующих элементов в том же столбце
  vector<int> jr;     // индексы первых ненулевых элементов строк
  vector<int> jc;     // индексы первых ненулевых элементов столбцов
};

void writeCompressedMatrix(ofstream& output, const CompressedMatrix& cm) {
  output << "AN: ";
  for (double val : cm.an)
  output << val << " ";
  output << "\nJR: ";

  for (int idx : cm.jr)
  output << idx << " ";
  output << "\nJC: ";

  for (int idx : cm.jc)
  output << idx << " ";
  output << "\nNR: ";

  for (int idx : cm.nr)
  output << idx << " ";
  output << "\nNC: ";

  for (int idx : cm.nc)
```

```
  output << idx << " ";
  output << endl;
}

void writeDecompressedMatrix(ofstream& output, const vector<vector<double>>&
    cm) {
  for (int i = 0; i < cm.size(); i++) {
    for (int j = 0; j < cm[i].size(); j++)
    output << cm[i][j] << " ";
    output << endl;
  }
}

CompressedMatrix compress(const vector<vector<double>>& original_matrix) {
  CompressedMatrix result;
  result.rows = original_matrix.size();
  result.cols = original_matrix[0].size();

  result.jr.resize(result.rows, -1);
  result.jc.resize(result.cols, -1);

  vector<vector<int>> rowLinks(result.rows);
  vector<vector<int>> colLinks(result.cols);

  //ненулевые элементы
  for (int i = 0; i < result.rows; i++) {
    for (int j = 0; j < result.cols; j++) {
      double val = original_matrix[i][j];
      if (val != 0) {
        int index = result.an.size();
        result.an.push_back(val);
        rowLinks[i].push_back(index);
        colLinks[j].push_back(index);
      }
    }
  }

  result.nr.resize(result.an.size(), -1);
  result.nc.resize(result.an.size(), -1);

  //строки
  for (int i = 0; i < result.rows; i++) {
    if (!rowLinks[i].empty()) {
      result.jr[i] = rowLinks[i][0];
      for (int k = 0; k < rowLinks[i].size(); k++) {
        int current_idx = rowLinks[i][k];
        int next_idx = (k + 1 < rowLinks[i].size()) ? rowLinks[i][k + 1] :
```

```
    rowLinks[i][0];
        result.nr[current_idx] = next_idx;
      }
    }
  }

  //столбцы
  for (int j = 0; j < result.cols; j++) {
    if (!colLinks[j].empty()) {
      result.jc[j] = colLinks[j][0];
      for (int k = 0; k < colLinks[j].size(); k++) {
        int current_idx = colLinks[j][k];
        int next_idx = (k + 1 < colLinks[j].size()) ? colLinks[j][k + 1] :
    colLinks[j][0];
        result.nc[current_idx] = next_idx;
      }
    }
  }

  return result;
}

vector<vector<double>> decompress(const CompressedMatrix& comp) {
  vector<vector<double>> result(comp.rows, vector<double>(comp.cols, 0.0));

  for (int i = 0; i < comp.rows; i++) {
    if (comp.jr[i] != -1) {
      int current = comp.jr[i];
      int start = current;

      do {
        int j = -1;

        for (int col = 0; col < comp.cols && j == -1; col++) {
          if (comp.jc[col] != -1) {
            int col_current = comp.jc[col];
            int col_start = col_current;
            do {
              if (col_current == current) {
                j = col;
                break;
              }
              col_current = comp.nc[col_current];
            } while (col_current != col_start);
          }
        }

```

```
        if (j != -1)
        result[i][j] = comp.an[current];

        current = comp.nr[current];
      } while (current != start);
    }
  }

  return result;
}

void printDecompressedMatrix(const CompressedMatrix& comp) {
  vector<vector<double>> matrix = decompress(comp);

  cout << "Распакованная матрица " << matrix.size() << "x" << matrix[0].size
   () << ":\n";
  for (int i = 0; i < matrix.size(); i++) {
    for (int j = 0; j < matrix[i].size(); j++)
    cout << matrix[i][j] << " ";
    cout << endl;
  }
}

CompressedMatrix transpose(const CompressedMatrix& A) {
  if (A.rows != A.cols)
  throw std::invalid_argument("Матрица должна быть квадратной");

  int n = A.rows;
  CompressedMatrix T;
  T.rows = n;
  T.cols = n;

  //столбец для каждого элемента в исходной матрице
  vector<int> element_col(A.an.size(), -1);

  //заполняем element_col
  for (int j = 0; j < n; j++) {
    if (A.jc[j] != -1) {
      int current = A.jc[j];
      int first = current;
      do {
        element_col[current] = j;
        current = A.nc[current];
      } while (current != first && current != -1);
    }
  }

```

```
vector<vector<pair<int, double>>> temp_data(n); // [строка_T] = [(столбец_T, значение)]

for (int i = 0; i < n; i++) {
  if (A.jr[i] != -1) {
    int current = A.jr[i];
    int first = current;
    do {
      int j = element_col[current]; // правильный столбец в исходке
      if (j != -1) {
        double value = A.an[current];
        //в транспонированной: строка j, столбец i
        temp_data[j].push_back({i, value});
      }
      current = A.nr[current];
    } while (current != first && current != -1);
  }
}

int nnz = A.an.size();
T.an.resize(nnz);
T.nr.resize(nnz);
T.nc.resize(nnz);
T.jr.resize(n, -1);
T.jc.resize(n, -1);

// заполнение T
int idx = 0;
for (int i = 0; i < n; i++) {
  if (!temp_data[i].empty()) {
    T.jr[i] = idx;

    for (int k = 0; k < temp_data[i].size(); k++) {
      T.an[idx] = temp_data[i][k].second;
      T.nc[idx] = temp_data[i][k].first;

      if (k < temp_data[i].size() - 1)
      T.nr[idx] = idx + 1;
      else
      T.nr[idx] = T.jr[i];
      idx++;
    }
  }
}

vector<vector<int>> col_elements(n);
for (int pos = 0; pos < nnz; pos++) {
```

```
    int col = T.nc[pos];
    col_elements[col].push_back(pos);
  }

  for (int j = 0; j < n; j++) {
    if (!col_elements[j].empty()) {
      T.jc[j] = col_elements[j][0];
      for (size_t k = 0; k < col_elements[j].size(); k++) {
        int elem_idx = col_elements[j][k];
        if (k < col_elements[j].size() - 1)
        T.nc[elem_idx] = col_elements[j][k + 1];
        else
        T.nc[elem_idx] = T.jc[j];
      }
    }
  }

  return T;
}

double arithmetic_mean(const CompressedMatrix& comp) {
  double sum = 0;
  for (double num : comp.an)
  sum += num;
  return sum / (comp.cols * comp.rows);
}
vector<vector<double>> readMatrixFromFile(const string& filename) {
  ifstream matrix_input(filename);
  if (!matrix_input)
  throw runtime_error("Ошибка: не удалось открыть файл " + filename);

  int rows, cols;

  if (!(matrix_input >> rows >> cols))
  throw runtime_error("Ошибка: неверный формат размеров матрицы");

  if (rows <= 0 || cols <= 0)
  throw runtime_error("Ошибка: размеры матрицы должны быть положительными");

  vector<vector<double>> matrix(rows, vector<double>(cols));
  string line;

  getline(matrix_input, line);

  for (int i = 0; i < rows; i++) {
    if (!getline(matrix_input, line))
    throw runtime_error("Ошибка: недостаточно строк в матрице");
```

```
    stringstream ss(line);
    for (int j = 0; j < cols; j++) {
      string token;
      if (!(ss >> token))
      throw runtime_error("Ошибка: недостаточно элементов в строке " + to_string(i + 1));

      try {
        size_t pos;
        double value = stod(token, &pos);

        if (pos != token.length())
        throw invalid_argument("нечисловой символ");

        matrix[i][j] = value;
      } catch (const invalid_argument& e) {
        throw runtime_error("Ошибка: нечисловое значение '" + token +
        "' в позиции [" + to_string(i + 1) + "][" + to_string(j + 1) + "]");
      } catch (const out_of_range& e) {
        throw runtime_error("Ошибка: число вне диапазона '" + token +
        "' в позиции [" + to_string(i + 1) + "][" + to_string(j + 1) + "]");
      }
    }
    string extra;
    if (ss >> extra)
    throw runtime_error("Ошибка: лишние элементы в строке " + to_string(i + 1));
  }

  matrix_input.close();
  return matrix;
}

int main() {
  ofstream output("result.txt");
  if (!output.is_open()) {
    cerr << "Не удалось открыть файл result.txt для записи" << endl;
    return 1;
  }

  try {
    vector<vector<double>> matrix = readMatrixFromFile("original_matrices.txt");

    output << "Исходная матрица " << matrix.size() << "x" << matrix[0].size() << ":\\n";
    writeDecompressedMatrix(output, matrix);
```

```
    output << endl;

    CompressedMatrix comp = compress(matrix);
    vector<vector<double>> decomp = decompress(comp);
    double mean = arithmetic_mean(comp);

    output << "Сжатая матрица:\\n";
    writeCompressedMatrix(output, comp);
    output << "\nРаспакованная матрица:\\n";
    writeDecompressedMatrix(output, decomp);
    output << "\nСреднее арифметическое матрицы: " << mean << endl;

    if (matrix.size() != matrix[0].size()) {
      output << "ОШИБКА: Матрица не квадратная (" << matrix.size()
      << "x" << matrix[0].size() << "), транспонирование невозможно\\n";
    }
    else {
      try {
        CompressedMatrix comp\_transposed = transpose(comp);
        vector<vector<double>> decomp\_transposed = decompress(
    comp_transposed);

        output << "Сжатая транспонированная матрица:\\n";
        writeCompressedMatrix(output, comp_transposed);
        output << "\nРаспакованная транспонированная матрица:\\n";
        writeDecompressedMatrix(output, decomp_transposed);
      } catch (const invalid_argument& e) {
        output << "ОШИБКА: " << e.what() << endl;
      }
    }

    output.close();
    cout << "Работа программы завершена. Результаты записаны в result.txt"
    << endl;

  } catch (const exception& e) {
    output << e.what() << endl;
    output.close();
    cerr << e.what() << endl;
    return 1;
  }
  return 0;
}
```

Листинг А.1 — Программная реализация разных алгоритмов умножения матриц